

Vocab: Fundamentals of Computer Systems (Quick Ref Guide - Midterm 1 (Spring 2024))

(Digital) Information: just sequences of binary (0's and 1's): True = 1, False = 0

Numbers: converted into binary form when "viewed" by computer

Characters: assigned a specific numerical value (ASCII standard)

Text: is a sequence of characters

Bit: a single binary digit (a '0' or a '1')

- Bit at the left is highest order (**a.k.a. most significant**) bit
- Bit at the right is lowest order (**a.k.a. least significant**) bit

Byte: a grouping of 8 bits, e.g., 10110010

Word: a grouping of bits that is *computer-architecture-dependent*

- The number of bits that the computer architecture can process at once
 - e.g., 64-bit word architectures expect data to be passed in (and returns outputs back out) in 64-bit groupings
 - The number of bits used by the architecture is called its **word size**
 - word size limits on the sizes of numbers they can deal with (in a single computation cycle)
 - # of bit sequences possible = 2^k (k = wordsize)

Overflow: result cannot fit within the wordsize constraint

Binary or Boolean Variable: a symbolic representation of a value that might be 0 or 1, e.g., X, Y, A, B

Complement (e.g., of a variable X): written $\bar{X}$: the opposite value of X

Literal: a boolean variable or its complement (e.g., X, $\bar{X}$, $\bar{Y}$)

Expression: a set of literals (possibly with repeats) combined with logic operations (and possibly ordered by parentheses)

Equation: expression1 = expression2

Function of (possibly several) variables: an equation where the lefthand side is defined by the righthand side

- can be described as the sum of **minterms**

Truth Table: Function output for all combinations of input variables (k variables $\rightarrow 2^k$ input combinations)

Depth: longest-gated path

- a k-input gate is a sequence of gates with depth $\log_2 k$
- 3 input gates each have depth 2

Product term: logical AND of literals (True only when all literals in the product term are True)

Sum term: logical OR of literals (True as long as some variables in the sum term are assigned to True)

Minterms: A product term in which all variables appear exactly once, either complemented or uncomplemented.

- Each minterm evaluates to 1 for exactly one assignment of values to all variables (one row in truth table), 0 for all others

Maxterms: "dual" of minterms; A sum term in which all variables appear once, either complemented or uncomplemented.

- Each maxterm evaluates to 0 for exactly one variable assignment, 1 for all others.

Implicant: product term (a "box" whose dimensions are powers of 2) comprised of minterms for which the function F evaluates to 1

Prime Implicant: An implicant not contained within another implicant (remember that implicant dimensions must be powers of 2!)

Essential Prime Implicant: a prime implicant that is the only prime implicant to cover some minterm.

Edge triggered: the output value of the FF depends only on the inputs at the instant in time when the clock transitions in value

Pulse triggered: the output value of the FF can depend on the sequence of input values during the interim of the pulse

Positive Edge: output value depends on the input during the 0-to-1 transition

Negative Edge: output value depends on the input during the 1-to-0 transition

Positive Pulse: Pulse Triggered and Leader latch active when C=1

Negative Pulse: Pulse Triggered and Leader latch active when C=0

Mod. Arith

X mod Y = remainder(X/Y) (i.e., a value between 0 and Y-1)

"Pinwheel" representation: move clockwise on the pinwheel for + numbers, counter-clockwise for - numbers (ex shown: mod 8)

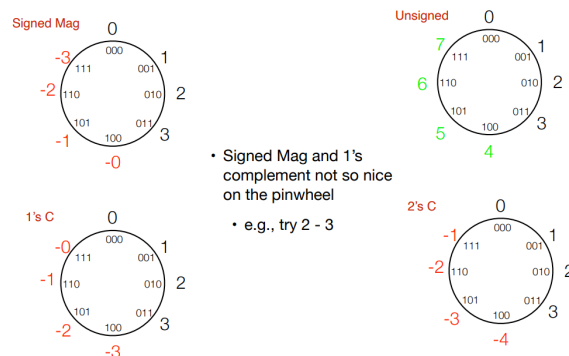
BAA (Unsigned)

1+1 = 0 (carry 1), 1+1+1 = 1 (carry 1), 1+0 = 1 (no carry)

restricted wordsize & overflow $\rightarrow$ limited_ws_ans = actual_ans mod 2^k

Unsigned Representation: 0 to $2^k - 1$ (e.g., k=8, 0 to 255)

Overflow: extra carry, can not be represented



Signed Magnitude

1st bit = sign (0+, 1-); remaining = magnitude

pos # in signed are same as unsigned

Representation: $-2^{k-1} + 1$ to $2^{k-1} - 1$ (e.g., k=8, -127 to 127 [2 ways to rep 0])

	Unsigned	Sign&Mag.	1s comp.	2s comp.
0 0 0	0	+0	+0	+0
0 0 1	1	+1	+1	+1
0 1 0	2	+2	+2	+2
0 1 1	3	+3	+3	+3
1 0 0	4	0	-3	-4
1 0 1	5	-1	-2	-3
1 1 0	6	-2	-1	-2
1 1 1	7	-3	0	-1

8 values

7 values,
2 zeroes

7 values,
2 zeroes

8 values,
1 zero

1's Comp

1st bit = sign (0+, 1-); remaining = magnitude

pos # in signed are same as unsigned

Negate: flip all bits

0000 0000 and 1111 1111 = 0

Representation: same as signed mag (but negative numbers are represented differently)

2's Comp

pos # in signed are same as unsigned

Negate: flip all bits and then (using binary add algorithm) add 1

- Ignore overflow (will only occur when negating 0)
- Negation works both ways (going + to - or - to +)
 - The exception: 1 followed by all 0's, e.g., 1000
 - value is -2^{k-1} for wordsize of k, e.g., k=4, value is -8
 - Note: the positive value of 2^{k-1} is not expressible with k bits in 2's complement form

Representation: -2^{k-1} to $2^{k-1} - 1$ (e.g., k=8, -128 to 127 [1 way to rep 0])

Subtraction: Negate subtrahend (bottom #), then add (BAA)

Overflow: final 2 carries differ

Q: Given an 8-bit wordsize, what is the value of 10001011?

A: Is it to be represented in Unsigned, Signed Magnitude, 1's complement or 2's complement?

• Unsigned: $128 + 8 + 2 + 1 = 139$

• Signed Mag: $-1 * (8 + 2 + 1) = -11$

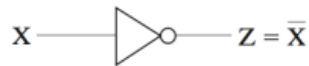
• 1's Complement: the negation of 01110100 = -116

• 2's Complement: the negation of 01110101 = -117

• Note: for a given set of bits, when # is negative, 2's complement is 1 less than 1's complement

NOT

x	$\bar{x}$
0	1
1	0



NOR

x	y	$z = \overline{x+y}$
0	0	1
0	1	0
1	0	0
1	1	0



AND

x	y	$x \cdot y$
0	0	0
0	1	0
1	0	0
1	1	1



NAND

x	y	$z = \overline{xy}$
0	0	1
0	1	1
1	0	1
1	1	0



NOT, AND, OR can each be implemented using only NAND gates

NOT, AND, OR can each be implemented using only NOR gates

$\bar{A} = A \text{ NAND } A$	$\bar{A} = A \text{ NOR } A$
$AB = \overline{\bar{A} \text{ NAND } \bar{B}}$	$A+B = \overline{\bar{A} \text{ NOR } \bar{B}}$
$A+B = \bar{A} \text{ NAND } \bar{B}$	$AB = \bar{A} \text{ NOR } \bar{B}$

OR

x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1



XOR

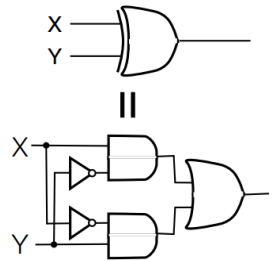
$$X \oplus Y = X\bar{Y} + \bar{X}Y$$

$$Y \oplus 0 = Y$$

$$Y \oplus 1 = \bar{Y}$$

As we XOR bits, the result “flips” when XOR’ing a 1, and stays the same when XOR’ing a 0

X	Y	$X \oplus Y$
0	0	0
0	1	1
1	0	1
1	1	0



Boolean Theorems

$$X + XY = X$$

$$X(X + Y) = X$$

$$X \oplus Y = X\bar{Y} + \bar{X}Y$$

$$Y \oplus 0 = Y$$

$$Y \oplus 1 = \bar{Y}$$

Consensus Theorem: $XY + \bar{X}Z + YZ = XY + \bar{X}Z$

DeMorgan’s: Removes larger bar

- Replace AND w/ OR, OR w/ AND
- Replace 1 w/ 0, 0 w/ 1
- Replace function F w/ $\bar{F}$, $\bar{F}$ w/ F
- Use to find complement of a function

Precedence: NOT (low/smaller bars first) -> AND -> OR

Complementing:

- to complement a function, compute its dual, then complement literals
 - 1. Swap ANDs and ORs
 - 2. “Flip” literals

Duals: Replace AND w/ OR, OR w/ AND, Replace 1 w/ 0, 0 w/ 1

For any boolean X ($X \in \{0,1\}$):

OR	AND	NOT	
$X+0 = X$	$X1 = X$		(identity)
$X+1 = 1$	$X0 = 0$		(null)
$X+X = X$	$XX = X$		(idempotent)
$X+\bar{X} = 1$	$X\bar{X} = 0$		(complementarity)
		$\bar{\bar{X}} = X$	(involution)
$X+Y = Y+X$	$XY = YX$		(commutativity)
$X+(Y+Z) = (X+Y)+Z$	$X(YZ) = (XY)Z$		(associativity)
$X(Y+Z) = XY + XZ$	$(X+Y)(X+Z) = X + YZ$		(distributive)
$\overline{X+Y} = \bar{X}\bar{Y}$	$\overline{XY} = \bar{X} + \bar{Y}$		(DeMorgan’s theorem)

Why?
We’ll see in 2 slides

Standard Forms (PoS & Sop)

Sum of products (SoP): OR of ANDs (OR of product terms)

- Order of precedence in this form: do all the ANDs before any ORs (don’t need parentheses)
- $F = \bar{Y} + \bar{X}Y\bar{Z} + XY$

Product of sums (PoS): AND of ORs (AND of sum terms)

- Order of precedence in this form: do all the ORs before any ANDs (Need parentheses)
- $X(\bar{Y} + Z)(X + Y + \bar{Z})$

Converting to SoP: Multiply through, then simplify

Converting from SoP to PoS: Complement, multiply through, complement (swap + and * ops, then flip literals)

Min/Maxterms

Sum of minterms form: The logical OR of all minterms for which $F = 1$.

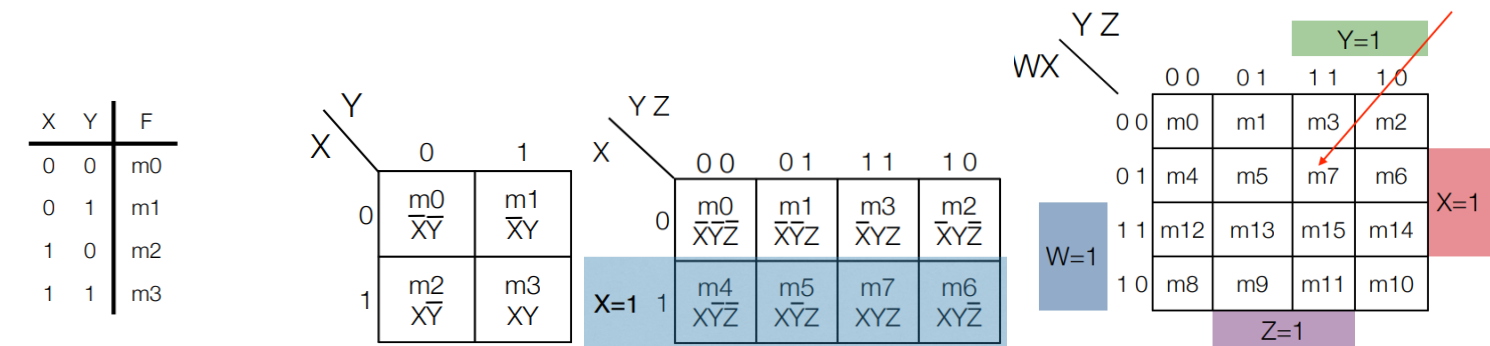
$$\sum m(1,2,3)$$

Product of maxterms form: The logical AND of all maxterms for which $F = 0$.

$$\prod M(0,4,5,6,7)$$

	F	F
Minterms (SOP)	$\sum m(F = 1)$	$\sum m(F = 0)$
Maxterms (POS)	$\prod M(F = 0)$	$\prod M(F = 1)$

K-Maps



A product term, which, viewed in a K-Map is a $2^i \times 2^j$ size “rectangle” (possibly wrapping around) where $i=0,1,2, j=0,1,2$

Using K-maps to Build Simplified Circuits:

- Step 1: Identify all PIs and essential PIs
- Step 2: Include all Essential PIs in the circuit
- Step 3: If any 1-valued minterms are uncovered by EPIs, choose PIs that are “big” and do a good job covering
- **Selection Rule:** choose the PIs that minimize overlap with one another and with EPIs

Finding $\overline{F}$: Find prime implicants corresponding to the 0s on a k-map

PoS expressions: Find $\overline{F}$ as SoP and then apply DeMorgan’s OR Cover the boxes with all “0”s - the sum term “0’s out” that box

Don’t Care Conditions: Marked as ‘X’ on K-Maps; can be optionally used to create largest implicants possible

Enabler Circuit

Enabler circuit has 2 inputs

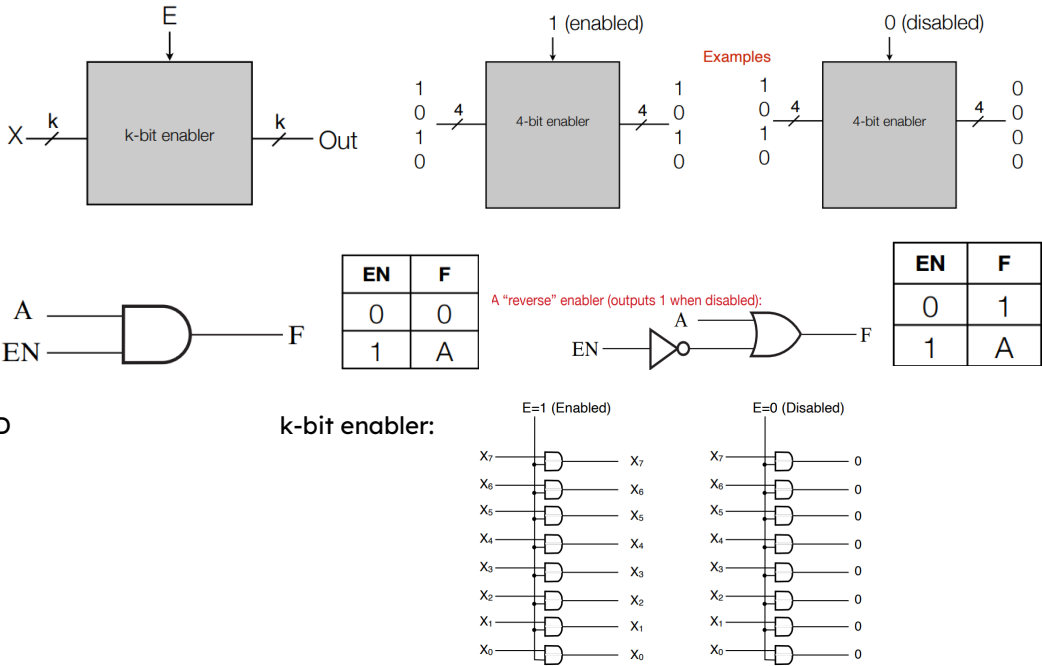
- data (can be k bits in parallel)
- enable/disable (i.e., on/off) input

Output:

- If E=1, output = data input
- If E=0, output = k 0’s

For both enabler circuits, output is “enabled” ($F=X$) only when input ‘ENABLE’ signal is asserted ($EN=1$).

Note the different output when DISABLED

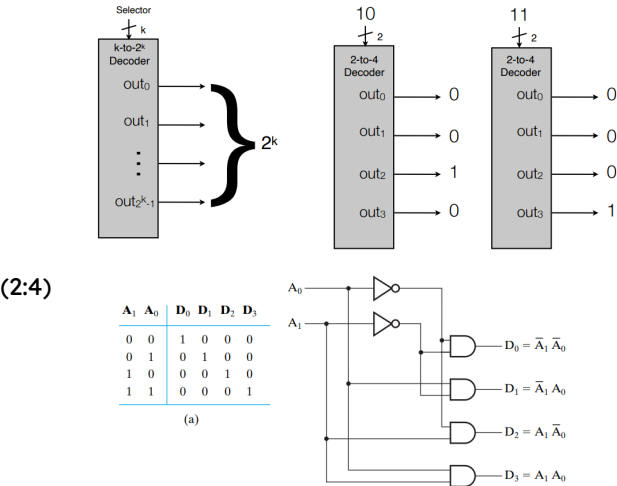
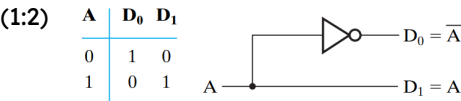


Standard Decoder Circuit

No “DATA” inputs, just a k-bit “selector” input

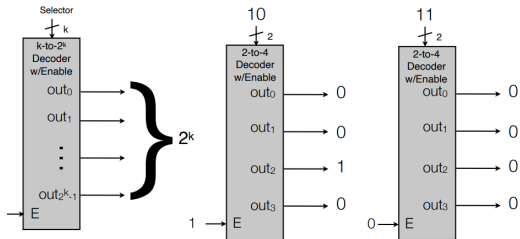
2^k 1-bit outputs

Selector input indicates as unsigned binary which output = 1, all other outputs = 0

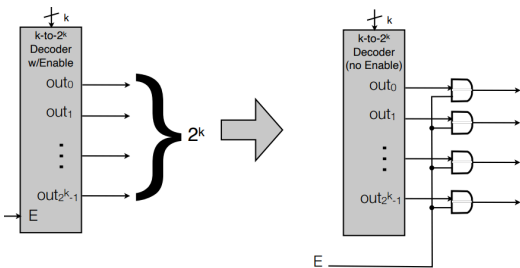


Decoder with Enable

- One additional input, E
- E=1: behaves like standard decoder
 - E=0: all outputs are 0

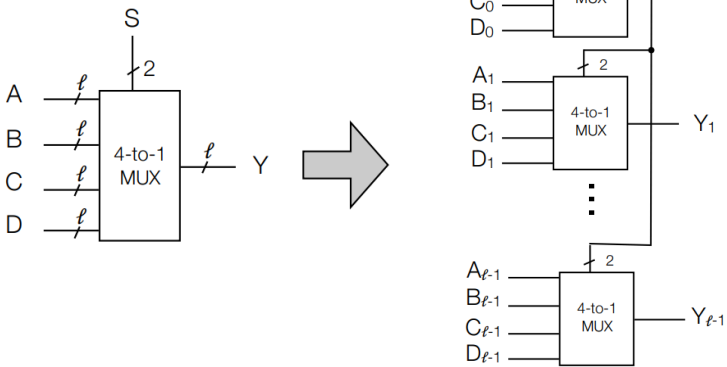
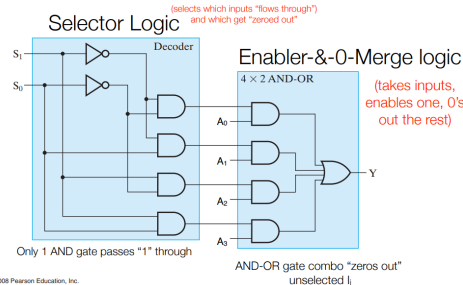
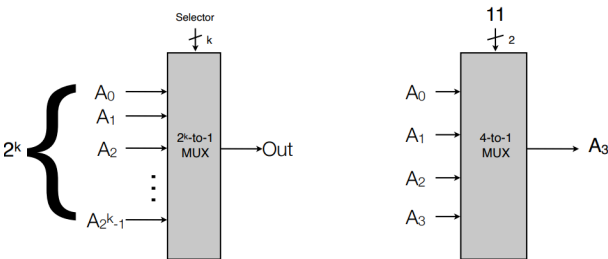


Decoder with Enable from Decoder:



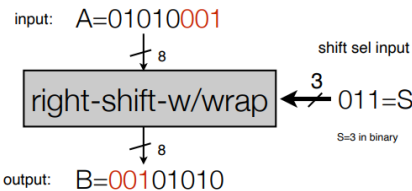
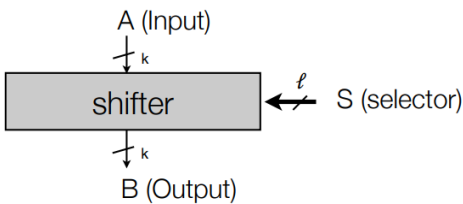
MUX Circuit

2^k 1-bit Data values enter as input
k-bit selector indicates **as unsigned binary** which data bit is output
≠bit 4-to-1 MUX: ≠bit quantities fed into a multiplexer, which selects one of those quantities
Implemented by feeding each of k bits into k separate 1-bit 4-to-1 MUX,
same selector bits fed into each MUX



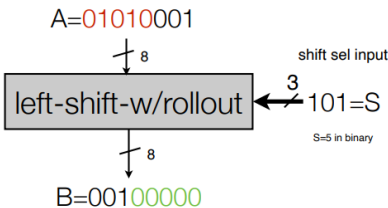
Shifter Circuit

k-bit data value entered as input
≠bit selector gives instructions how to shift
(Barrel) Shifter Circuit: ≠ = log₂k-bit selector indicates (unsigned binary) how far bits should be shifted



← k=8, right shifter with “wraparound”, asked to shift by 3 bits

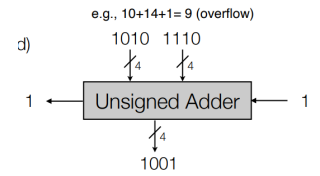
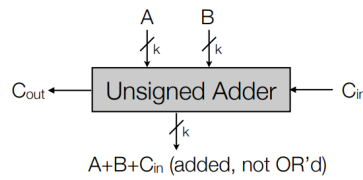
k=8, left shifter with “rollout”, asked to shift by 5 bits (fill missing bits with 0’s)



(Unsigned) Adder Circuit

Two k-bit data values, A and B, and 1 bit Cin (carry-in) entered as input

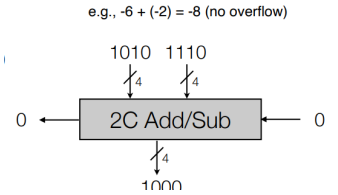
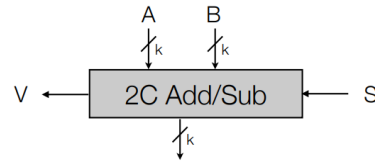
Output is k-bit A+B + Cin (unsigned binary addition), and 1-bit Cout (carry-out), i.e., the overflow



(Signed 2's-C) Adder/Subtractor Circuit

Two k-bit data values, A and B, and 1 bit S=0 for add, S=1 for subtract

Output is k-bit A+B when S=0, A-B when S=1 (2's comp binary add/sub), and 1-bit overflow, V

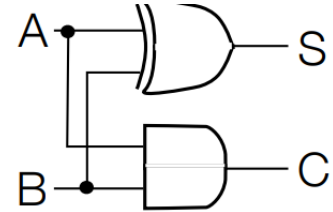


Addition: The Half-Adder

-Addition of 2 bits: A & B produces a summand (S) and carry (C)

- $S = A \oplus B$
- $C = AB$

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

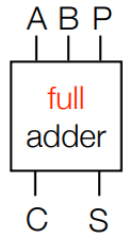
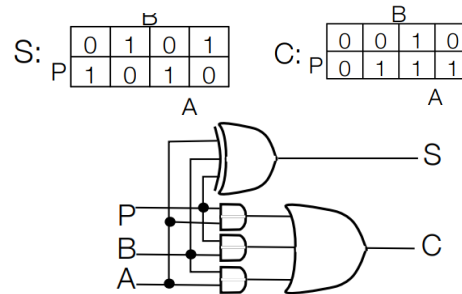


Full Adder

-Takes as input 2 digits (A&B) and a previous carry (P)

- $S = A \oplus B \oplus P$
- $C = AB + AP + BP$

P	A	B	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



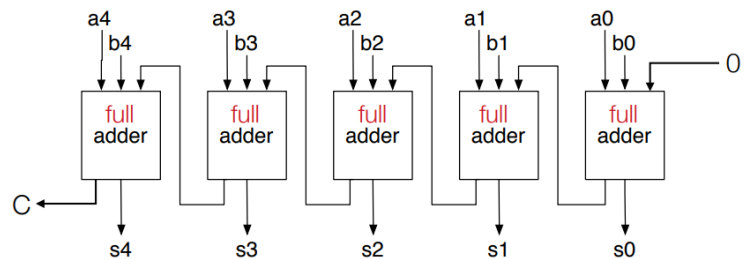
5-bit Ripple Carry Adder

-Computes $a_4a_3a_2a_1a_0 + b_4b_3b_2b_1b_0$

-Each full adder's has depth 2 (inputs pass through 2 gates to reach output)

-Full adder that computes s_i cannot "converge" to its result until previous full adder computes carry

-The longest depth in a k-bit ripple carry adder is $3k$ (each carry is pairwise ANDs followed by 3-way OR, and feeds into next computation)



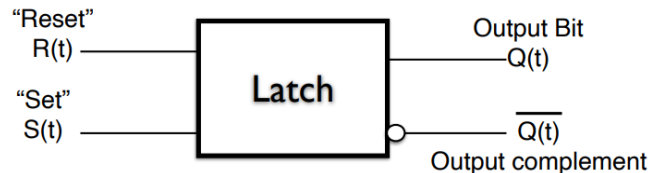
Latches

R=0, S=1, write 1 into the latch

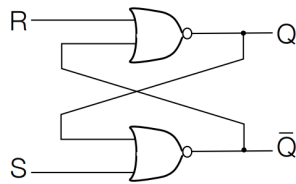
R=1, S=0, write 0 into the latch

R=S=0, latch "holds" its value

R=S=1: do not use!!!



SR Latch:



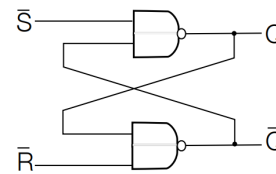
R	S	Q	$\bar{Q}$
0	0	Q	$\bar{Q}$
0	1	1	0
1	0	0	1

No change
Set (Q=1)
Reset (Q=0)

$\bar{S}\bar{R}$ Latch:

Latch constructed of cross-coupled NAND gates

$\bar{R}$ and $\bar{S}$ have reverse behavior of SR latch, so R and S have same behaviors



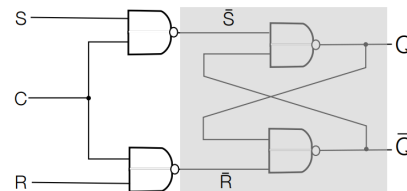
$\bar{R}$	$\bar{S}$	Q	$\bar{Q}$
0	0	1	1
0	1	0	1
1	0	1	0
1	1	Q	$\bar{Q}$

SR Latch with Control:

(inner latch is $\bar{S}\bar{R}$ latch)

C=0, $\bar{S}\bar{R}$ latch receives $\bar{S}=1$, $\bar{R}=1$, values "hold"

C=1, first set of NAND gates invert S & R inputs to $\bar{S}$ & $\bar{R}$

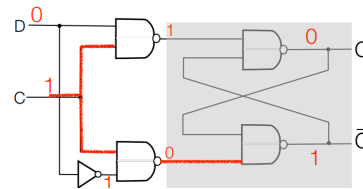


C	R	S	Q	$\bar{Q}$
0	X	X	Q	$\bar{Q}$
1	0	0	Q	$\bar{Q}$
1	0	1	1	0
1	1	0	0	1
1	1	1	1	1

D Latch with Control:

With the control (C), no reason to ever have $S=R \cdot C=0$, latch holds value,

C=1, Q=D



C	D	Q	$\bar{Q}$
0	X	Q	$\bar{Q}$
1	0	0	1
1	1	1	0

Flip Flops

Change once per (full) clock cycle

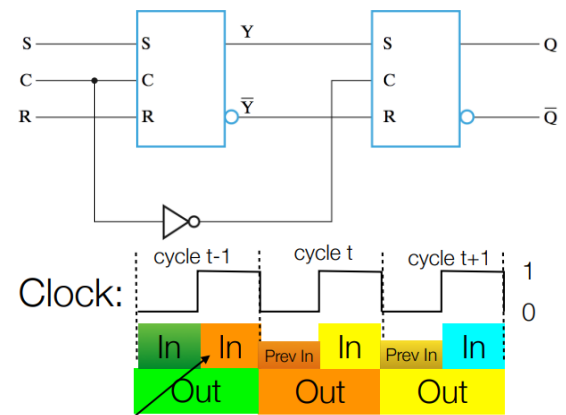
- A full clock cycle is one iteration of clock going down then up (or up then down)
- Flip Flop (as built so far) "releases" new value each time clock goes 1→0 • What is released for the i-th cycle was set during (bottom half) of i-1st cycle

SR Flip-Flop:

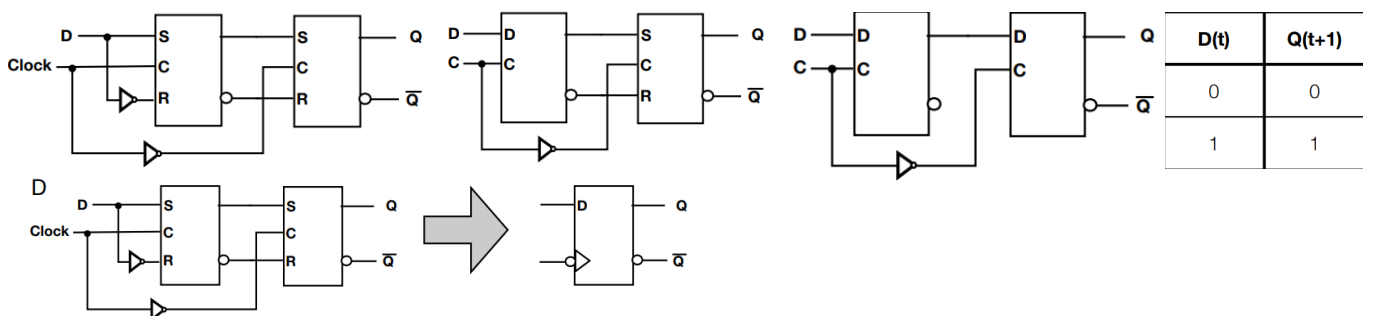
2 SR Latches w/ control

C (Control) is fed a clock pulse (alternates between 0 and 1 with fixed period)

- C=1: Leader latch "on", Follower latch "off"
 - New S & R inputs read into leader
 - Previous Q values still emitted (not affected by new S&R inputs)
- C=0: Leader latch "off", Follower latch "on"
 - Changing S & R inputs has no effect on Lead (or Follower) latch
 - S&R inputs from last time C=1 stored safely in Leader and transferred into Follower



D Flip-Flop:

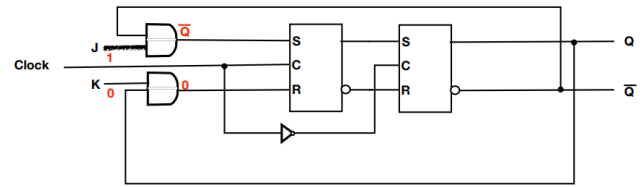


D(t)	Q(t+1)
0	0
1	1

JK Flip Flop from SRFF:

- $J=1, K=0$; **sets** the JK F.F.: $Q(t+1) = 1$
- $J=0, K=1$; **resets** the JK F.F.: $Q(t+1) = 0$
- $J=0, K=0$; **stays same**
- $J=1, K=1$; **complements** the JK F.F.: $Q(t+1) = \bar{Q}(t)$

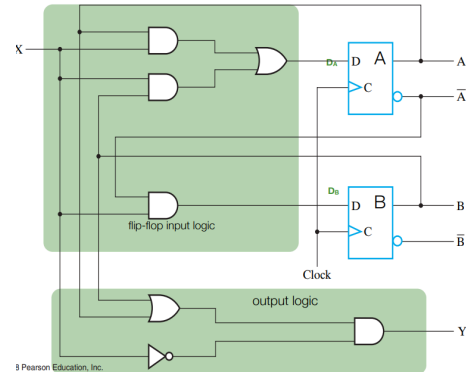
J(t)	K(t)	Q(t+1)
0	0	Q(t)
0	1	0
1	0	1
1	1	$\bar{Q}(t)$



Sequential Circuit:

What happens in Clock cycle t?

- **X(t)**: an arbitrary binary input
 - can assume stays fixed for entire clock cycle
- **A(t)**: output of top D flip-flop (fixed)
- **B(t)**: output of bottom D flip-flop (fixed)
- **Y(t)**: output of circuit is function of X(t), A(t), B(t)
- **A(t+1)**: gets set (as a function of X(t), A(t), B(t))
- **B(t+1)**: gets computed (as a function of X(t), A(t))



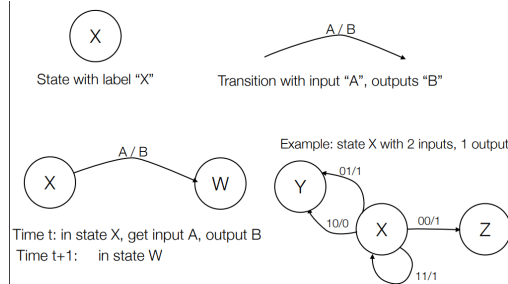
State Machines/JK Flip Flop Design:

A: input

B: output

X: State (can be represented in binary /eg. 00, 01, 10, etc)

States or inputs can be multiple bits



A _{cur}	A _{next}	J,K
0	0	0,0 or 0,1 (0,X)
0	1	1,0 or 1,1 (1,X)
1	0	0,1 or 1,1 (X,1)
1	1	0,0 or 1,0 (X,0)

To Create Flip Flop (Example

Table/Circuit):

AB: 2 bit state (A is higher, B is Lower)

Create table containing:

- Input
- Current states(seperate bits)
- Next States
- J and K for each bit
- and your Output

Create KMaps for Js, Ks, and Output

A and B are D-Flip Flops in your circuit

A _{cur}	B _{cur}	In	A _{next}	J _A	K _A	B _{next}	J _B	K _B	Out
0	0	0	1	1	X	0	0	X	0
0	0	1	0	0	X	1	1	X	0
0	1	0	0	0	X	0	X	1	1
0	1	1	1	1	X	0	X	1	0
1	0	0	0	X	1	1	1	X	0
1	0	1	0	X	1	0	0	X	1
1	1	0	X	X	X	X	X	X	X
1	1	1	X	X	X	X	X	X	X

$$J_A = B\bar{I} + \bar{B}\bar{I} \quad K_A = 1 \quad J_B = \bar{A}\bar{I} + A\bar{I} \quad K_B = 1$$

